

# Attention and Transformer Fundamentals

*A practical educational document for learning Retrieval-Augmented Generation (RAG)*

**Purpose:** This document introduces the attention mechanism and the Transformer architecture in a way that is useful for understanding modern language models and RAG systems.

## Learning goals

- Understand why attention was introduced and what problem it solves.
- Understand queries, keys, and values (Q, K, V).
- Understand scaled dot-product attention and the softmax step.
- Understand multi-head attention, positional information, and Transformer blocks.
- Connect Transformer concepts to language models and RAG.

## Core idea

Attention lets a model decide which parts of an input are important when producing a representation for a particular token. Instead of treating every word as equally relevant, the model learns different weights for different tokens.

## Simple example

Consider: "The animal did not cross the road because **it** was tired." To understand "it", a useful model should pay attention to "animal". Attention provides a mechanism for learning such relationships.

# 1. Why Attention Matters

Older sequence models often processed text step by step. Recurrent architectures such as RNNs and LSTMs can model sequence order, but long-distance relationships can be difficult to preserve. Attention allows a token to directly consider other tokens in the sequence.

## Contextual representation

A word does not have one fixed meaning in every sentence. For example, the word “bank” can refer to a financial institution or the side of a river. A contextual representation changes according to surrounding words.

| Sentence                         | Important context |
|----------------------------------|-------------------|
| I deposited money at the bank.   | money, deposited  |
| The boat reached the river bank. | boat, river       |

## Attention as information selection

- Input tokens are represented as vectors.
- The model compares a token with other tokens.
- Relevant tokens receive larger attention weights.
- The weighted information is combined into a new representation.

## Why this matters for LLMs

Large language models use attention repeatedly across many layers. This helps them model syntax, semantic relationships, references, and long-range dependencies. RAG systems do not replace this ability; they provide additional external context that the model can attend to while generating an answer.

## 2. Queries, Keys, and Values

Self-attention is commonly described using three learned projections: Query (Q), Key (K), and Value (V). They are produced from the token representations.

| Component | Intuition                          | Role                                                       |
|-----------|------------------------------------|------------------------------------------------------------|
| Query     | What am I looking for?             | Represents the current token's information need.           |
| Key       | What information do I contain?     | Used to compare with a query.                              |
| Value     | What information should I pass on? | Content aggregated after attention weights are calculated. |

### Analogy: library search

Imagine a library visitor has a question. The question is the Query. Each book has searchable metadata that acts like a Key. The actual useful material inside the selected books acts like the Values. Attention calculates which keys best match the query and then combines the corresponding values.

### Important distinction

A Query is not itself the final answer. It is a representation used to decide which information is relevant. Similarly, Keys are not the information that is ultimately copied into the output; Values provide the content that is aggregated.

### Matrix view

For a sequence of token vectors  $X$ , learned matrices transform  $X$  into  $Q$ ,  $K$ , and  $V$ . In simplified form:  $Q = XW_Q$ ,  $K = XW_K$ , and  $V = XW_V$ .

### 3. Scaled Dot-Product Attention

*The standard attention calculation is often written as:*

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_K})V$$

**The process has three conceptual stages.**

- **Score:** Compute similarities between queries and keys using a dot product.
- **Scale:** Divide by  $\sqrt{d_K}$  to keep values at a useful numerical scale.
- **Normalize:** Apply softmax so scores become weights that sum to approximately 1, then use them to combine values.

#### **Tiny numerical intuition**

Suppose a query produces scores [2, 1, 0] for three tokens. Softmax turns these raw scores into positive weights, with the first token receiving the largest weight. The output becomes a weighted combination of the three value vectors.

#### **Why scaling?**

When vector dimensions are large, dot products can become large in magnitude. Large softmax inputs can produce very sharp distributions and unstable gradients. Scaling by  $\sqrt{d_K}$  helps control this effect.

#### **Key lesson**

Attention is not simply “finding the nearest word.” It is a learned, weighted information aggregation mechanism operating over vector representations.

## 4. Multi-Head Attention

One attention operation can learn some relationships, but different relationships may be useful at the same time. Multi-head attention runs several attention mechanisms in parallel.

### Conceptual example

- One head may focus on grammatical relationships.
- Another may capture relationships between a pronoun and its antecedent.
- Another may focus on local phrase structure.
- Another may capture broader semantic relationships.

Each head has its own learned projections for Q, K, and V. The outputs of the heads are concatenated and passed through another learned projection.

### Why multiple heads?

Different representation subspaces can encode different patterns. Multiple heads give the model several ways to examine the same sequence.

| Single head                     | Multiple heads                         |
|---------------------------------|----------------------------------------|
| One attention pattern at a time | Several attention patterns in parallel |
| Limited representation subspace | Different learned subspaces            |
| One view of token relationships | Multiple views of relationships        |

### Practical connection

When reading a technical document, different pieces of context may matter for different reasons: a definition, a previous sentence, an entity name, or a condition. Transformer attention is designed to represent such relationships.

## 5. Positional Information and Transformer Blocks

Attention by itself does not automatically encode the order of tokens. A sequence such as “dog bites man” is different from “man bites dog,” so the model needs information about position.

### Positional encoding

Transformers can inject positional information into token representations. Modern architectures use several approaches, including learned positional embeddings and rotary positional representations.

### Typical Transformer block

- Input token representations enter the block.
- Self-attention mixes information across tokens.
- A residual connection helps preserve and combine information.
- Layer normalization stabilizes representations.
- A feed-forward network transforms each position.
- Another residual connection and normalization complete the block.

### Encoder vs decoder intuition

| Architecture    | Typical use                                                             |
|-----------------|-------------------------------------------------------------------------|
| Encoder         | Build rich representations of an input; common in understanding tasks.  |
| Decoder         | Generate tokens autoregressively; common in generative language models. |
| Encoder-decoder | Transform one sequence into another, such as translation.               |

Modern LLMs used for generation are commonly decoder-style Transformer architectures, although many variations exist.

## 6. Attention in Language Models

During language generation, a decoder model predicts the next token from previous context. Causal masking prevents the model from using future tokens that would not yet be available.

### Causal attention

If a sentence has tokens 1, 2, 3, and 4, the representation for token 3 can attend to tokens 1–3 but not token 4. This preserves the left-to-right generation objective.

### Training vs generation

- **Training:** many positions can often be processed in parallel using a causal mask.
- **Generation:** tokens are produced sequentially, and previously computed states can be cached for efficiency.

### Why context length matters

Attention operates over the available context window. If a relevant fact is outside the model's accessible context, the model cannot directly attend to it.

### Connection to RAG

RAG addresses one important limitation: instead of expecting the model to have every useful document in its parameters, a retrieval system can select relevant external passages and place them into the model's context before generation.

## 7. Attention, Embeddings, and RAG

| Concept           | Main question                                                                      |
|-------------------|------------------------------------------------------------------------------------|
| Embedding         | How can an item such as text be represented as a vector?                           |
| Similarity search | Which stored vectors are most related to this query vector?                        |
| Attention         | Which available context should the model focus on while computing representations? |
| RAG               | How can external information be retrieved and supplied to a generator?             |

### Important distinction

Embeddings and attention are not the same thing. Embeddings create vector representations used for comparison or downstream processing. Attention dynamically combines information among representations according to learned relevance scores.

### Example RAG flow

- A user asks: “What is multi-head attention?”
- The question is converted into an embedding.
- A vector database searches for semantically similar chunks.
- Relevant chunks from a PDF are retrieved.
- The chunks are placed into the LLM prompt as context.
- The LLM uses its Transformer layers, including attention, to interpret the supplied context and generate an answer.

### Final takeaway

Attention is a core mechanism behind modern Transformer models. Understanding Q/K/V, weighted aggregation, masking, and contextual representations will make the behavior of LLM-based RAG systems much easier to understand.